

SeIM: In-Memory Acceleration for Approximate Nearest Neighbor Search

Chaoqiang Liu*, Dan Chen[†], Yu Huang*, Wenjing Xiao[‡], Haifeng Liu*, Yi Zhang*,
Huize Li[†], Xiaofei Liao*, and Hai Jin*

*National Engineering Research Center for Big Data Technology and System/Services Computing Technology and System Lab/Cluster and Grid Computing Lab, Huazhong University of Science and Technology, Wuhan, China

[†]School of Computing, National University of Singapore, Singapore

[‡] Guangxi Key Laboratory of Multimedia Communications and Network Technology, Guangxi University, Nanning, China

chqliu@hust.edu.cn, danchen@nus.edu.sg, yuh@hust.edu.cn, wenjingx@gxu.edu.cn,
{hfliu, yizh}@hust.edu.cn, huizeli@nus.edu.sg, {xfliao, hjin}@hust.edu.cn

Abstract—Approximate nearest neighbor search (ANNS) is crucial in many applications to find semantically similar matches for user queries. Especially with the development of *large language models* (LLMs), ANNS is becoming increasingly important in *retrieval-augmented generation* (RAG). An in-depth analysis of ANNS reveals that its diverse operations, from extensive memory access to intensive sorting, are key performance bottlenecks, imposing significant strain on both the memory system and computing resources. Based on these observations, we present SeIM, a hierarchical in-memory architecture to accelerate ANNS. SeIM is designed to accommodate the diverse operational characteristics of ANNS. Specifically, SeIM offloads highly parallel memory-bound operations to the memory bank level and introduces a unified *execution model* to reuse hardware units, requiring only lightweight modifications to standard DRAM architecture. Additionally, SeIM places compute-bound sorting operations, which require cross-unit data access, at the memory controller level and employs an *adaptive transmission filtering* technique to reduce unnecessary data transfers and processing during sorting. Our evaluation shows that SeIM achieves 268 $\times$, 22 $\times$, and 5 $\times$ higher throughput, 306 $\times$, 59 $\times$, and 4 $\times$ lower latency, and 3081 $\times$, 287 $\times$, and 2 $\times$ higher power efficiency than state-of-the-art CPU-, GPU-, and ASIC-based ANNS solutions.

Index Terms—processing-in-memory, DIMM, ANNS.

I. INTRODUCTION

Approximate nearest neighbor search (ANNS) is a fundamental technique for retrieving vectors similar to a query vector in vector space, widely used in deep learning [1] and information retrieval [2] systems. Traditional *large language models* (LLMs), which rely on static training data, often struggle with real-time updates, leading to contextual misunderstandings and inaccurate responses. The *retrieval-augmented generation* (RAG) [3], [4] addresses this limitation by combining ANNS-enabled vector databases with LLMs, incorporating external knowledge to improve the performance of generative models. Additionally, recommender systems like YouTube [5] and search engines like Google [6] use ANNS to improve suggestions and search results.

This work is supported by the National Key Research and Development Program of China (No. 2023YFB4503400), the National Natural Science Foundation of China and Research Grants Council (NSFC/RGC) Joint Research Scheme (No. 62461160333), and the National Research Foundation, Singapore, under its Competitive Research Programme Award NRF-CRP23-2019-0003. Dan Chen is the corresponding author.

ANNS can be classified into four main categories: quantization-based [7], graph-based [8], hash-based [9], and tree-based [10]. Among these, quantization-based ANNS has emerged as the industry's first choice [11] for large-scale search due to their high performance and low memory footprint. In quantization-based ANNS, the search process begins by filtering out the cluster centroids closest to the query vector. Next, a distance table is constructed using the codebook, and the compressed vectors within the filtered clusters are scanned. The distance to the query vector can be quickly obtained for each compressed vector by looking up the distance table. This process mainly involves three types of operation: *vector computation*, *distance sorting*, and *lookup-based computation*.

We conduct an in-depth analysis of the search process using the mainstream ANNS library FAISS [12] and identify three key findings. First, search performance is constrained by multiple phases collectively, rather than a single phase. Therefore, optimizing search performance requires comprehensive improvements across the entire process, rather than focusing on a specific phase. Second, there are significant differences in the operational intensity of various operations: *vector computation* and *lookup-based computation* are highly memory-bound and susceptible to frequent data movement between memory and processing units, whereas *distance sorting* is primarily limited by computational capacity. Lastly, these three operations exhibit a mixed computation pattern: *vector computation* and *lookup-based computation* are highly parallelizable but differ in their processing for each dimension, *distance sorting* requires sequential aggregation of the results from all subtasks, complicating parallelization efforts.

Previous studies [13]–[15] have demonstrated that *processing-in-memory* (PIM) architectures hold significant potential for accelerating memory-bound applications. However, applying existing PIM solutions to quantization-based ANNS presents several challenges. First, the search process involves both memory-bound and compute-bound operations, posing challenges for existing PIM architectures to fully support the search process. Second, while current PIM accelerators [16], [17] optimized for *general matrix-vector multiplication*

(GEMV) excel at *vector computation*, they lack optimization for *lookup-based computation*, which demands distinct hardware designs and may introduce additional overhead. Therefore, new designs are needed to accelerate quantization-based ANNS.

In this work, we present SeIM, a high-performance and energy-efficient in-memory accelerator for quantization-based ANNS. SeIM's innovation lies in its hierarchical acceleration design, which addresses the distinct characteristics of search operations and delivers full functionality at a low cost. On the one hand, we integrate computation logic into memory to support both *vector computation* and *lookup-based computation*, requiring only lightweight modifications to DIMMs based on standard DRAM technology. On the other hand, we aggregate computation results in the memory controller and efficiently handle sorting operations. With higher power and area budgets compared to DIMMs, the memory controller supports the integration of more sufficient computational capabilities. To the best of our knowledge, SeIM is the first solution to leverage PIM technique for accelerating quantization-based ANNS.

In particular, SeIM first deploys custom processing units within each DRAM bank, specifically designed to support *vector computation* and *lookup-based computation*. By employing all-bank mode, it maximizes internal bandwidth utilization and parallelism. Additionally, SeIM introduces a unified *execution model* for these offloaded-to-memory operations, enabling logical reuse and minimizing hardware resource overhead. Second, SeIM incorporates hardware priority queues within the memory controller to efficiently handle *distance sorting*. SeIM further employs an *adaptive transmission filtering* technique to reduce memory transfers caused by invalid distance values. Finally, SeIM adopts a heterogeneous computing programming model, similar to CUDA/SYCL [18], to enhance development efficiency and improve the user experience.

We list our contributions as follows:

- We perform an in-depth characterization of quantization-based ANNS, identifying key performance bottlenecks and motivating the design of in-memory acceleration.
- We introduce SeIM, the *first* in-memory accelerator for quantization-based ANNS, which addresses the diverse requirements of various operations in the search process through a hierarchical PIM architecture.
- We further propose two key optimizations: (I) a unified *execution model* to reduce hardware resource overhead for in-memory computations, and (II) an *adaptive transmission filtering* technique to decrease unnecessary data transfers for sorting.
- Experimental results show that SeIM achieves significant improvements in both performance and energy efficiency compared to existing software and hardware solutions.

II. BACKGROUND AND MOTIVATION

A. Quantization-based ANNS

Product Quantization (PQ). PQ is a widely employed technique for compressing high-dimensional vectors [7]. To mitigate compression errors, this algorithm begins by splitting

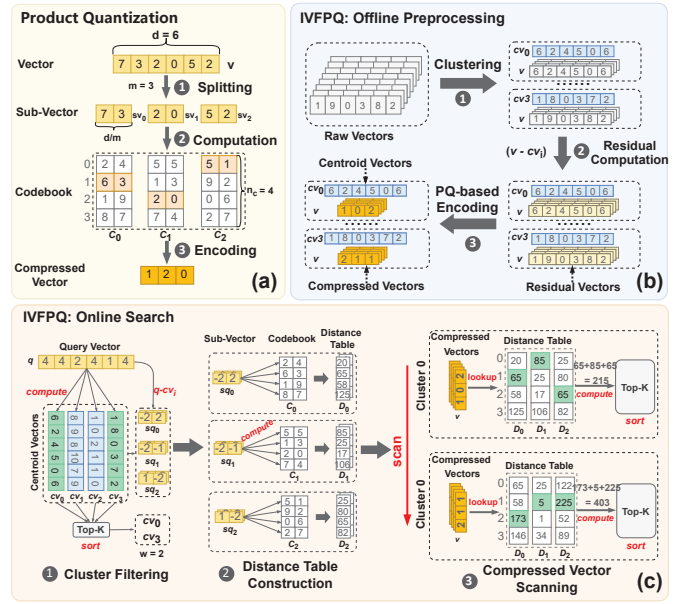


Fig. 1. An illustrative example of quantization-based ANNS (L2 distance) the input vector v , of dimension d , into m distinct sub-vectors sv_i (where $0 \leq i < m$), each with a dimension of d/m (1). For each sub-vector sv_i , the algorithm identifies the index of its nearest vector within the corresponding codebook C_i (2). Consequently, the final vector v is encoded as a concatenation of m indices (3). Fig. 1(a) illustrates an example of encoding a six-dimensional vector v with PQ. After encoding, v can be represented by three indices $\{1, 2, 0\}$.

PQ-based ANNS. Many popular ANNS libraries typically integrate the *inverted file index* (IVF) with PQ to reduce the number of compressed vectors that need to be scanned [7], a technique known as IVF-PQ. Fig. 1(b) illustrates the offline preprocessing process of IVF-PQ for database vectors. First, it clusters the database vectors (1). Then, it computes the residuals of each vector relative to its respective cluster centroid (2). Finally, it encodes all residual vectors utilizing product quantization (3). After encoding is complete, the vector database will store the trained centroid vectors and compressed vectors.

Since preprocessing is an one-time operation, our primary focus is on the online search process. In ANNS, commonly used distance metrics between vectors include *euclidean distance* (L2) and *inner product distance* (IP). Fig. 1(c) shows the process of online search in IVF-PQ using the L2 distance. The initial phase is *cluster filtering* (CF). It selects the w nearest centroid vectors to the query vector q (1). The subsequent phase is *distance table construction* (DTC). The algorithm computes the residual between the query vector q and the centroid vector of the selected cluster, then splits it into m sub-vectors and calculates the distance between each sub-vector and its corresponding codebook. Note that the computation process of IP distance is different from that of L2 distance (2) [11]. The final phase is *compressed vector scanning* (CVS). It processes the compressed vectors within the selected w clusters. This is executed by using each dimension of a compression vector as an index to retrieve and accumulate

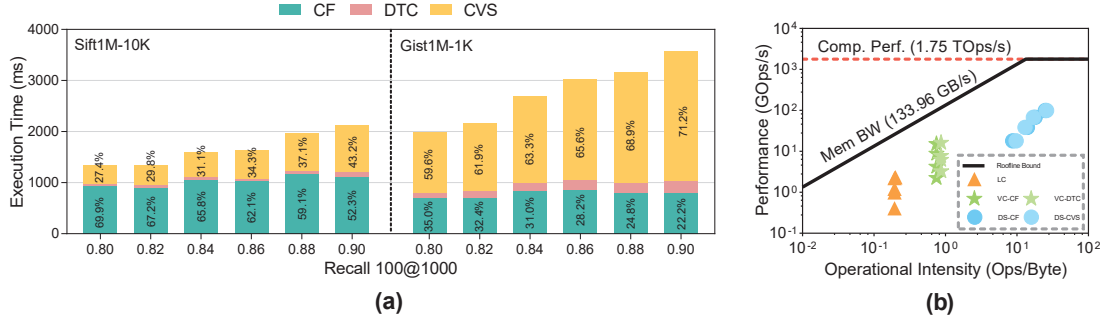


Fig. 2. (a) Search breakdown for different phases across datasets with varying recall (0.84, 0.86, 0.88, 0.90) and (b) roofline model analysis of ANNS, sweeping batch size

data from the corresponding distance table. After obtaining the distance values, the algorithm must also sort them to identify the top- k most similar vectors (③).

B. Characterizations of Quantization-based ANNS

We characterize, FAISS [12], the popular ANNS library built by Facebook, on the CPU to identify performance bottlenecks and optimization opportunities. Experimental details are in §IV-A.

Performance Analysis. Fig. 2(a) delineates the execution time distribution across the three phases of the search process. Notably, *cluster filtering* and *compressed vector scanning* account for over 90% of the total time, indicating that optimizing these two phases is crucial to increase performance. We further analyze the key operations involved within these three phases. There are three key operations including: *vector computation* (VC), *distance sorting* (DS), and *lookup-based computation* (LC). During the *cluster filtering* phase, operations include *vector computation* and *distance sorting*; *Distance table construction* is primarily focused on *vector computation*; and *compressed vector scanning* consists of *lookup-based computation* and *distance sorting*.

Roofline Analysis. Fig. 2(b) depicts the roofline model data points for three key operations under various query conditions. The model indicates that *vector computation* and *lookup-based computation* are memory-bound, whereas *distance sorting* operation is compute-bound. Taking SIFT1M [19] as an example, the original vectors are 128-dimensional (4 bytes per element), compressed to 64-dimensional (1 byte per element), with a search for the top-100 nearest neighbors. For *vector computation*, this entails a 512-Byte DRAM read and 383 arithmetic operations (128 SUB, 128 MUL, 127 ADD), translating to an operational intensity of 0.75 operations per Byte. *Lookup-based computation* operation reads the 64-Byte compressed vector, retrieves 64 4-Byte distance values, and finally performs 63 additions with an intensity of 0.19 operations per Byte. *Distance sorting* operation, receiving a 4-Byte distance, involves updating a priority queue with up to 100 comparisons, reflecting an intensity of 25 operations per Byte.

Computation Pattern Analysis. *Vector computation* and *lookup-based computation* share similarities and key differences. Both lack dependencies between tasks, allowing them to be divided into independent subtasks for parallel processing.

However, *vector computation* performs element-wise operations between two vectors followed by accumulation, while *lookup-based computation* uses vector dimensions as indices to retrieve and accumulate results from a lookup table. In contrast, *distance sorting* has a completely different pattern. It aggregates results from the other two operations to update a priority queue. Due to the dependencies in updates, distance sorting cannot be split into independent parallel subtasks.

C. Processing-in-memory Acceleration for Quantization-based ANNS

Why Choose PIM? PIM is a key technique proposed to accelerate the memory-bound operation. By integrating the compute unit in memory, PIM technique can significantly shorten the data transfer path between memory and processing units [13]–[15]. Quantization-based ANNS involves extensive memory access, making it suitable for PIM with high bandwidth and low latency capabilities. However, achieving this goal remains challenging. First, the characteristics of search operations vary, requiring more specific designs than existing PIM solutions. *Vector computation* and *lookup-based computation* are memory-bound and highly parallel, making them ideal for lower memory levels like the bank level. In contrast, *distance sorting* is compute-bound and requires aggregating results from the other two operations, making it better suited for higher memory levels. Second, while current PIM solutions [16], [17] for matrix-vector multiplications (GEMV) focus on *vector computation* at the memory bank level, they lack optimizations for *lookup-based computation*. Supporting *lookup-based computation* would require new hardware designs, potentially increasing overhead.

How to Design? To address these challenges, we propose a hierarchical PIM design. For *vector computation* and *lookup-based computation*, we deploy processing units at the memory bank level, maximizing the utilization of internal memory bandwidth. Additionally, we design a unified execution model and computation logic for these two operations, enabling resource sharing and minimizing hardware modifications, while maintaining compatibility with commercial DIMM architectures. For *distance sorting*, which requires aggregating results from the other two operations and involves computation-intensive sorting tasks, we extend the memory controller with dedicated hardware for sorting acceleration. Furthermore,

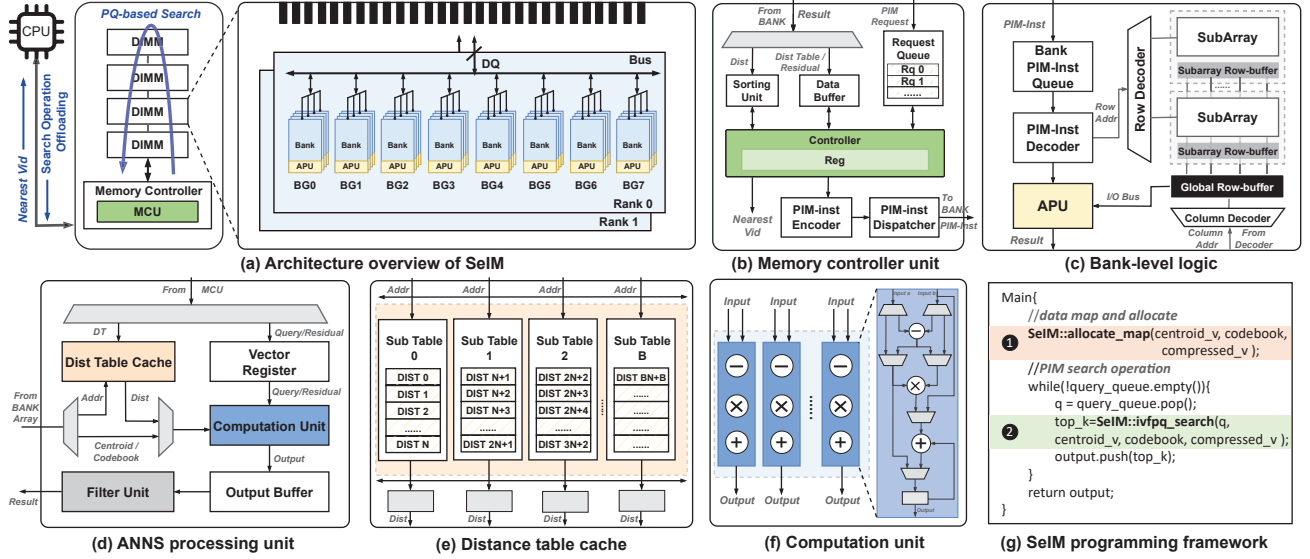


Fig. 3. The SeIM architecture

we propose an adaptive filtering mechanism to reduce the transmission and sorting of invalid results.

III. SEIM ARCHITECTURE

In this section, we introduce SeIM, a DIMM-based hierarchical PIM architecture designed to accelerate quantization-based ANNS. Although SeIM is implemented using DIMM in this work, its design principles are equally applicable to HBM/HMC.

A. Architecture Overview

Fig. 3(a) depicts the overall architecture of SeIM. Unlike conventional memory systems, SeIM integrates a *memory controller unit* (MCU) within the memory controller, which manages the entire data flow and performs sorting operation. Additionally, components such as *ANNS processing unit* (APU) are deployed on each bank within the DIMMs to support diverse computing tasks. These APUs can simultaneously respond to computing instructions at the bank level, thereby exploiting DRAM's bank-level parallelism and significantly enhancing computational efficiency. Notably, SeIM maintains compatibility with conventional memory devices, as all original interfaces remain unchanged.

B. Hardware Design

Memory Controller Unit (MCU). The detail of the MCU is illustrated in Fig. 3(b). The MCU comprises several components: a controller, a request queue, an encoder, a dispatcher, a data buffer, and a sorting unit. The primary function of the request queue is to temporarily store PIM requests from the host. The controller is responsible for activating either the sorting unit or the encoder based on the current progress. The encoder generates a PIM instruction for each PIM request, which is then transferred by the dispatcher to the corresponding banks within the DIMMs. To overcome C/A bandwidth limitation, SeIM utilizes the compression technique to encode all DDR commands, tags, and other relevant information in a

single PIM instruction. The data buffer stores the distance table or residual vectors generated during the search process. The sorting unit is implemented as a hardware priority queue [20] based on the binary heap structure to handle all sorting tasks.

Bank-level Logic. As shown in Fig. 3(c), SeIM integrates the APU directly within each DRAM bank to enable efficient near-memory computation. In addition to the APU, SeIM incorporates an instruction queue and instruction decoder into each bank. These components are responsible for receiving and decoding PIM instructions from the memory controller. This design enables each bank to independently execute its assigned computational tasks in parallel.

ANNS Processing Unit (APU). As depicted in Fig. 3(d), the core component of the APU is the computation unit. Fig. 3(f) details the design of the computation unit, which supports both *vector computation* and *lookup-based computation*. Additionally, for *vector computation*, the unit supports both IP distance and L2 distance metrics, which cover most search scenarios. The APU also includes a distance table cache for storing the distance table, a vector register for caching query and residual vectors, an output buffer for holding computation results, and a filter unit that supports the adaptive transmission filtering technique, as introduced in § III-D.

Distance Table Cache (DTC). Based on the execution model described in § III-C, the APU simultaneously loads multiple compression vectors of the same dimension, which serve as addresses for lookup in the distance table. To enhance read bandwidth, the distance table cache is divided into multiple independently addressable tables, as illustrated in Fig. 3(e). In an ideal scenario, the input addresses will be distributed across different tables, allowing the distance table cache to read multiple distance values concurrently.

C. Data Layout and Execution Model

The tasks offloaded to memory (*vector computation* and *lookup-based computation*) share similar computation pattern. Therefore, we propose a unified execution model to reuse

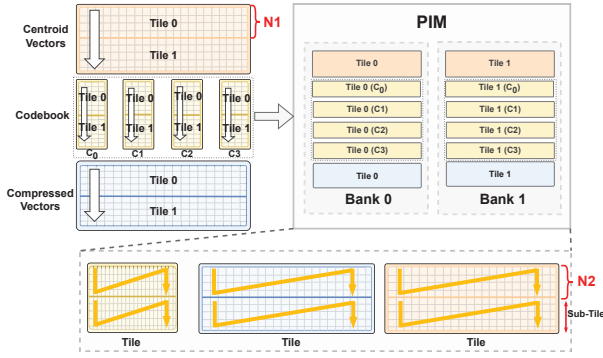


Fig. 4. Data layout and execution model

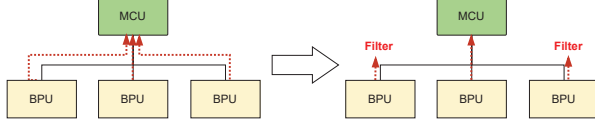


Fig. 5. Adaptive transmission filtering

hardware logic and maximize bank-level parallelism. Meanwhile, the data involved in the computations, such as the centroid vectors, codebook, and compressed vectors, all follow a consistent data layout scheme. The data layout scheme and execution model are shown in Fig. 4.

Across banks, we divide the three types of data into equally sized tiles along the row dimension, and each tile is assigned to a different bank. Each bank processes identical tasks with nearly the same amount of data, ensuring load balancing between processing units. During the search phase, memory operates in all-bank mode, where each bank processes its assigned tasks in parallel.

Within bank, for *lookup-based computation*, processing the compressed vector tiles in row-major order would require storing the entire distance table in the on-chip memory, which could result in significant area overhead. On the other hand, using column-major order would also necessitate a large buffer to store the intermediate results for all rows. To address this issue, we further subdivide the tile along the row dimension into smaller sub-tiles, with the size determined by the buffer capacity. Within each sub-tile, we store and process the compressed vectors in column-major order. To simplify hardware design, the same processing model is applied to *vector computation*.

One particular scenario is when using L2 distance for searching, where we need to compute the residuals between the query vector and each selected centroid vector. To reduce unnecessary row activations, we make a copy of the centroid tiles in each bank with an entirely row-major layout, which only consumes a few extra megabytes of memory.

D. Adaptive Transmission Filtering

The sorting unit must aggregate data from all banks, which may introduce new bottlenecks. We find that most of the distance values read into the memory controller are discarded and do not participate in the actual queue updates. To address this, we design a filtering mechanism, as shown in Fig. 5, which periodically transmits the maximum distance value from

TABLE I
SEIM HARDWARE CONFIGURATION
AND PARAMETERS

DRAM Specification	
DDR5-4800 MHz, 4 Channels $\times$ 1 DIMM $\times$ 2 Ranks	
64-entry RD/WR queue, FR-FCFS	
DRAM Timing Parameters	
tRCD=40, tCL=40, tRP=40, tBL=8, tCCD_S=8, tCCD_L=12	
tRAS=76, tFAW=32, tRC=116	
SeIM Specialized Units	
1 APUs per Bank, 1.06KB Cache/Buffer per APU	
1MB Buffer per MCU	

TABLE II
DATASET INFORMATION

Dataset	#Vector	#Dim	Distance Metric	Domain
Deep1M	10^6	96	L2	Image
Sift1M	10^6	128	L2	Text
Gist1M	10^6	960	L2	Image
Deep1B	10^9	96	L2	Image
Sift1B	10^9	128	L2	Text
TTI1B	10^9	128	IP	Image&Text

the priority queue to each APU and saves it. When the APU calculates new results, it compares them to the saved maximum distance value; if the new result exceeds this value, it is discarded directly without being transferred to the memory controller for sorting. This approach not only reduces the amount of data transmitted in memory but also saves sorting time overhead.

E. Programming Model and Execution Flow

SeIM adopts a heterogeneous programming model to enable PIM functionality. Fig. 3(g) illustrates the programming framework of SeIM. Before executing the search, SeIM allocates and maps data—including centroid vectors, codebooks, and compressed vectors—into corresponding memory regions, following the data layout strategy described in § III-C, while mapping indices to physical addresses. When a search operation is initiated, the host receives a batch of query requests. For each query, SeIM activates the PIM kernel to generate PIM requests that are then executed in memory. Upon completion of the search, the top- k neighbor IDs of all query requests in the batch are returned from the memory to the host through the DRAM interface.

IV. EVALUATION

A. Experimental Setup

SeIM Settings. We implement an internal cycle-accurate simulator based on Ramulator [21] for evaluating the performance of SeIM. The basic hardware configuration of SeIM is summarized in Table I. We synthesize the special hardware units using Synopsys Design Compiler [22] under a 32nm technology library at the frequency of 500MHz. The energy consumption, area, and access latency of all buffers in SeIM are estimated by CACTI [23].

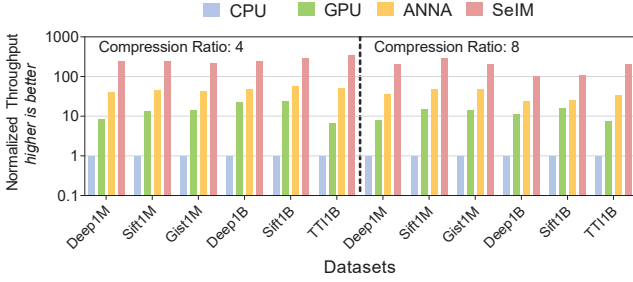


Fig. 6. Throughput comparison (recall 100@1000)

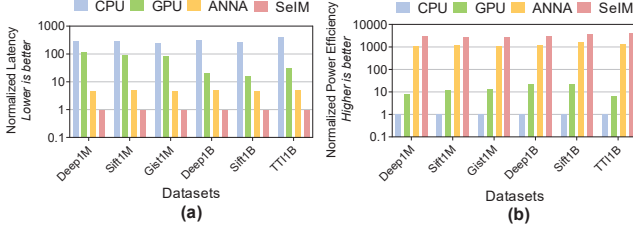


Fig. 7. (a) Latency and (b) power efficiency comparison

Datasets and Metrics. We pick four representative real-world datasets, Sift [19], Gist [19], Deep [24], and TTI [24], with different data scales. The dataset details are shown in Table II. We compare the performance of the different solutions by measuring the *throughput* (QPS) and *latency* (ms) of the queries with the same *recall* ($R@K$), which refers to the fraction of the true R nearest neighbors included in the K search results. Additionally, we take *energy efficiency* (QPS/Watt) into account to evaluate the balance between performance and energy consumption.

Baselines. We compare SeIM to CPU, GPU, and ASIC baselines. For the CPU baseline, we choose a 14-core, dual-socket Xeon Gold 5117 CPU with 512GB of DDR4 memory, utilizing 56 threads to search in parallel during testing. The GPU baseline runs on an Nvidia V100 with 32GB of HBM2 memory. The performance and power consumption are measured by Intel RAPL and nvprof tools. We further compare SeIM with the state-of-the-art PQ-based ANNS accelerator ANNA [11]. Note that all methods are based on the most popular PQ-based ANNS library FAISS [12], and all parameters in the algorithm are identical.

B. Experimental Results

Throughput Comparison. Fig. 6 illustrates the throughput comparison of SeIM compared to CPU, GPU, and ANNA at different compression rates. Compared with CPU, GPU, and ANNA, SeIM achieves on average $188.07\times$, $16.99\times$, and $5.09\times$ throughput improvement at compression rate of 8, and $268.76\times$, $22.92\times$, and $5.59\times$ at compression rate of 4, respectively. The performance improvement of SeIM comes from several factors. (1) SeIM utilizes near-memory processing to fully utilize in-memory bandwidth and dramatically reduce data transfer distances. (2) SeIM customizes a highly parallelized near-memory processing unit for searching and pipelines of the data path. (3) SeIM introduces optimizations such as *adaptive Transmission Filtering*, which further reduces data transfer as well as processing.

TABLE III
SEIM HARDWARE OVERHEAD

SPU Hardware Overhead of One DIMM	
Power (mW):	73.46 (CU), 320.71 (Buffer&Cache), 394.17 (Total)
Area (mm ²):	0.185 (CU), 0.262 (Buffer&Cache), 0.447 (Total)
MCU Hardware Overhead	
Power (mW):	71.35 (SU), 2565.70 (Buffer), 2637.05 (Total)
Area (mm ²):	0.103 (SU), 2.098 (Buffer), 2.201 (Total)

Latency Comparison. We further evaluate the average query latency of SeIM against the baselines when the compression rate is 4. As shown in Fig. 7(a), SeIM reduces the query latency by $306.42\times$, $59.03\times$, and $4.85\times$ compared to CPU, GPU, and ANNA. Moreover, SeIM can still process similarity searches in milliseconds for data at a billion scale, which indicates that SeIM is well suited for latency-constrained scenarios to meet strict SLA constraints.

Power and Area. Table III reports the area and power cost of SeIM. In total, SeIM adds 0.447mm^2 for area and 0.39W for power consumption per DIMM. In comparison, a single commercial DIMM chip typically consumes about 10W of power and 3500mm^2 of area, respectively, and the additional overhead introduced by SeIM is negligible. For the upper MCU, the area and power consumptions are 2.63W and 2.201mm^2 , respectively.

Power Efficiency Comparison. Fig. 7(b) provides the power efficiency comparison between SeIM and baselines. On average, SeIM is $3081.37\times$, $287.02\times$, and $2.49\times$ more power efficient than CPU, GPU, and ANNA, respectively. This increased efficiency can be attributed to two main factors: first, minimizing the distance of data movement reduces energy consumption associated with data transfers; second, the highly parallel and customized computational architecture significantly accelerates search time.

Impact of SeIM Optimization. We compare the throughput of SeIM with and without *Adaptive Transmission Filtering*. On average, at a compression rate of 4, SeIM with *Adaptive Transmission Filtering* achieves $2.13\times$ throughput compared to SeIM without *Adaptive Transmission Filtering*. Similarly, at a compression rate of 8, this leads to an additional speedups of $1.94\times$.

V. CONCLUSION

In this paper, we present SeIM, a high-performance and energy-efficient in-memory acceleration architecture for ANNS, designed to fully support the entire search process. We conduct a comprehensive analysis of mainstream ANNS tasks, identifying key performance bottlenecks and architectural requirements. Based on this, we propose a hierarchical PIM design that fully leverages internal memory bandwidth to accelerate memory-bound computations and uses controller-level hardware to accelerate compute-bound sorting. Extensive evaluations show that SeIM significantly outperforms state-of-the-art CPU-, GPU-, and ASIC-based solutions in both performance and energy efficiency.

REFERENCES

- [1] Y. Cao, H. Qi, W. Zhou, J. Kato, K. Li, X. Liu, and J. Gui, "Binary hashing for approximate nearest neighbor search on big data: A survey," *IEEE Access*, vol. 6, pp. 2039–2054, 2017.
- [2] M. Flickner, H. S. Sawhney, J. Ashley, Q. Huang, B. Dom, M. Gorkani, J. Hafner, D. Lee, D. Petkovic, D. Steele, and P. Yanker, "Query by image and video content: The QBIC system," *IEEE Computer*, vol. 28, no. 9, pp. 23–32, 1995.
- [3] P. S. H. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proceedings of the 34th Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [4] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J. Lespiau, B. Damoc, A. Clark, D. de Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. W. Rae, E. Elsen, and L. Sifre, "Improving language models by retrieving from trillions of tokens," in *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022, pp. 2206–2240.
- [5] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for youtube recommendations," in *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*, 2016, pp. 191–198.
- [6] Statista, "Worldwide visits to google.com." [Online]. Available: <https://www.statista.com/statistics/268252/web-visitor-traffic-to-googlecom/>
- [7] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.
- [8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [9] L. Gong, H. Wang, M. Ogihara, and J. Xu, "idec: Indexable distance estimating codes for approximate nearest neighbor search," *Proceedings of the VLDB Endowment*, vol. 13, no. 9, pp. 1483–1497, 2020.
- [10] A. Arora, S. Sinha, P. Kumar, and A. Bhattacharya, "Hd-index: Pushing the scalability-accuracy boundary for approximate knn search in high-dimensional spaces," *Proceedings of the VLDB Endowment*, vol. 11, no. 8, pp. 906–919, 2018.
- [11] Y. Lee, H. Choi, S. Min, H. Lee, S. Beak, D. Jeong, J. W. Lee, and T. J. Ham, "ANNA: specialized architecture for approximate nearest neighbor search," in *Proceedings of the 28th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 169–183.
- [12] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 03, pp. 535–547, 2021.
- [13] S. H. Lee, S. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, S. O, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware architecture and software stack for PIM based on commercial DRAM technology : Industrial product," in *Proceedings of the 48th Annual ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.
- [14] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. S. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: in-memory accelerator for bulk bitwise operations using commodity DRAM technology," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 273–287.
- [15] F. Devaux, "The true processing in memory accelerator," in *Proceedings of IEEE Hot Chips 31 Symposium (HCS)*, 2019, pp. 1–24.
- [16] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. N. Vijaykumar, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 372–385.
- [17] J. Park, B. Kim, S. Yun, E. Lee, M. Rhu, and J. H. Ahn, "Trim: Enhancing processor-memory interfaces with scalable tensor reduction in memory," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 268–281.
- [18] K. Group, "Sycl overview." [Online]. Available: <https://www.khronos.org/sycl/>
- [19] M. Aumueeller, E. Bernhardtsson, and A. Faitfull, "Benchmarking nearest neighbors." [Online]. Available: <https://github.com/erikbern/ann-benchmarks>
- [20] R. Bhagwan and B. Lin, "Fast and scalable priority queue architecture for high-speed network switches," in *Proceedings of the IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies (Cat. No. 00CH37064)*, vol. 2, 2000, pp. 538–547.
- [21] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible DRAM simulator," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, 2016.
- [22] Synopsys, "Design compiler." [Online]. Available: <https://www.synopsys.com/>
- [23] H. laboratories, "Cacti 6.0: A tool to model large caches." [Online]. Available: <https://www.hpl.hp.com/techreports/2009/HPL-2009-85.pdf>
- [24] Y. Research, "Benchmarks for billion-scale similarity search." [Online]. Available: <https://research.yandex.com/blog/benchmarks-for-billion-scale-similarity-search>